Maren

INTERNAL, MAREN FINANCE

Maren Finance
Work Summary

What was built, what has been verified, what has not, and what to do next.

DOCUMENT	DATE	VERSION	PREPARED FOR
MRN-007	15 August 2026	1.1	Sele

DOCUMENT	DATE	CLASSIFICATION	VERSION
MRN-007	15 August 2026	Internal, Maren Finance	1.1

Maren Finance — Work Summary

What was built, what has been verified, what has not, and what to do next.

CONTENTS

1 What this document is 3

2 What exists 4

2.1 Contracts 4

2.2 Why MarenRelayHub exists 5

2.3 The relayer 6

2.4 Landing page 6

2.5 iOS app 6

2.6 Documents 6

3 What is verified 6

3.1 The contract suite 7

3.2 The fork tests 8

3.3 Contract sizes 8

3.4 The headline number 8

3.5 The chain facts the thesis rests on 9

3.6 Gas 9

3.7 The applications 10

3.8 The document set 10

4 What is not verified, and what is not done 11

4.1 Matsnet is deployed; mainnet is not 11

4.2 The relayer is not merged 12

4.3 MarenEarn's swap path has venues, but is not wired yet 12

4.4 The pooled vault carries real systemic risk 12

4.5 Everything is pre-audit 13

4.6 Two environment limitations worth recording 13

5 Two findings that did not survive verification 14
5.1 The dRPC claim 14
5.2 The mis-citation 15
6 Upstream contributions, prepared and not submitted 15
7 What to do next 16
8 The shape of it 17

1

What this document is

This is the seventh document in the Maren set and the only one written inward rather than outward. The other six argue a case to the Mezo ecosystem team. This one records what is actually in the repository, what has been proven about it, and what has not, so that nothing in the first six has to be taken on trust when someone asks where a number came from.

It is organised as four questions:

1. **What exists.** The components, their state, and where they live.
2. **What is verified.** Numbers with the command that produces them.
3. **What is not.** The gaps, the risks, and the two claims that were withdrawn.
4. **What to do next.** In priority order.

Parts of it will inform what goes to Mezo. It is not itself the reply, that draft is `docs/research/OUTREACH-REPLY.md`, and it should stay a separate artefact.

2

What exists

Six components. Four are merged to `main`, one is an open pull request, and one is this document set.

COMPONENT	PATH	STATE
Contracts	`packages/contracts`	Merged (PR #3)
Landing page	`apps/web`	Merged (PR #2)
iOS app	`apps/ios`	Merged
SDK	`packages/sdk`	Merged
Relayer	`apps/relayer`	**PR #1, open, not merged**
Documents	`docs/src`	Seven sources, built to branded A4 PDFs

2.1

Contracts

Solidity 0.8.28, Foundry, OpenZeppelin v5.4.0. Five contracts are deployed; a sixth, `MarenForwarder`, is the hub's base class and is deliberately never deployed on its own.

CONTRACT	WHAT IT IS
`MarenVault`	The pooled trove. 0.001 BTC floor, 180% borrow floor, 140% internal liquidation.
`MarenPay`	MUSD payments: direct, by handle, permit, requests, links, split.
`MarenRegistry`	`@handle` to address, commit-reveal, reverse resolution.
`MarenEarn`	ERC-4626 over the MUSD Stability Pool.
`MarenRelayHub`	The forwarder actually deployed. Atomic, signed-fee relaying.

`MarenVault` is the differentiator and it is also the only contract that can lose a user money through a Maren design failure. That asymmetry is why it holds most of the test weight and all of the audit scope.

2.2

Why `MarenRelayHub` exists

It exists because of a specific finding, not because a hub seemed tidy. Both obvious ways to charge a relay fee are broken, and the second one is broken in a way that looks correct.

A trailing `uint256 relayFee` parameter on every function was rejected. The signature covers it, but no contract does anything with it, consent to a number with no settlement attached. It also drags relayer economics into `MarenPay` and `MarenVault`, which have no business knowing about them.

Batching a fee leg and a work leg through `executeBatch` was rejected because it does not do what its name implies. OpenZeppelin's `ERC2771Forwarder.executeBatch` is **not atomic for inner-call failures**. The `atomic` flag governs *validation* only, untrusted target, expired deadline, bad signature. The inner call is a raw `call` whose `success` is recorded and then used for nothing except deciding whether to refund `value`. A batched fee leg therefore settles even when the work leg reverts, including with `refundReceiver == address(0)`. This is asserted directly by `test_executeBatchIsNotAtomicForInnerCallFailures`, and it is why `MarenRelayHub` disables `executeBatch` outright with `BatchExecutionDisabled()`.

What the hub does instead: one EIP-712 `RelayedCall` signature covers the target, the exact calldata, the gas, the nonce, the deadline, **and** `feeToken` / `feeAmount` / `feeRecipient`. The hub takes the fee and performs the call in the same frame, bubbling any inner revert, so the fee and the work settle together or neither does.

Five properties are asserted directly rather than inferred:

PROPERTY	ENFORCED BY
Atomic	fee then call in one frame; the inner revert is bubbled
Fee is inside the signed payload	all three fee fields are members of the EIP-712 struct
Exactly one trusted forwarder	the hub *is* a `MarenForwarder`; `Deploy.s.sol` wires exactly one
The end user survives to the target	ERC-2771 suffix, with `_isTrustedByTarget` checked first
Target and selector allowlist	per-`(target, selector)`, on the fee path and the fee-free `execute`

The fourth is the one most likely to fail silently, so it is asserted against `test/mocks/SenderProbe.sol`, which records both `_msgSender()` and the raw `msg.sender` and observes that the first is the end user while the

second is the hub. That is a direct observation, not an inference drawn from a payment that happened to move the right balances.

2.3

The relayer

`apps/relayer` is a Hono service that verifies an EIP-712 signature, simulates the call by `eth_call`, and submits it while paying the BTC gas. It is **open as PR #1 and has not been merged**, and the reason is stated in section 4.2.

2.4

Landing page

`apps/web` is Next.js 15, fully static, with three shader usages and nothing above the fold that a reduced-motion user cannot opt out of.

2.5

iOS app

`apps/ios` is Expo with `expo-router`: onboarding, wallet setup, portfolio, vault, pay, earn, activity, settings, and scan-to-pay. All position math, number formatting, and protocol constants come from `@maren/sdk` and are real. Chain reads and transaction submission are mocked, and the README enumerates exactly which is which, the split is that everything deciding what a user *believes about their money* is real, and only the transport is stubbed.

2.6

Documents

Seven Markdown sources in `docs/src` become branded A4 PDFs through headless Chrome: Markdown to HTML, printed in three parts, a zero-margin cover so the gradient reaches the trim edge, a body with a running footer, and a contents page whose page numbers are measured from a first pass and stamped into a second. `pdf-lib` stitches them and sets the metadata. Fonts are embedded as base64 so a PDF renders identically on a machine that has never heard of Satoshi.

3

What is verified

Everything in this section was re-executed on 15 August 2026, in the worktree this document was written in, and the command is given so it can be run again.

3.1

The contract suite

```bash
cd packages/contracts && forge test
```

236 tests passed, 0 failed, across ten suites, with 8 skipped, the skips are the fork tests, which skip rather than fail when no RPC is configured, so that a public testnet cannot break CI.

SUITE	TESTS
`MarenPay.t.sol`	61
`MarenRegistry.t.sol`	52
`MarenVault.t.sol`	48
`MarenRelayHub.t.sol`	26
`MarenEarn.t.sol`	20
`MarenForwarder.t.sol`	12
`RelayerIntegration.t.sol`	6
`MarenVault.invariant.t.sol`	6
`MarenVault.fuzz.t.sol`	5
Total	**236**

The six invariants each complete **4,096 calls across 64 runs with zero reverts**: collateral is fully backed, the interest index never decreases, nobody withdraws more than they deposited, the per-account ledger sums to the total, the reserve is backed by real MUSD, and the sum of per-user debt covers the trove's own debt.

That last one is the load-bearing property of the pooled design. It is the machine-checked form of the sentence "the vault's books cannot drift below the protocol's books", and every rounding decision in the contract is oriented to keep it true.

3.2

The fork tests

```bash
MEZO_TESTNET_RPC_URL=https://rpc.test.mezo.org \
  forge test --match-path 'test/fork/*'
```

8 tests passed, 0 failed, against the real MUSD contracts deployed on matsnet, `BorrowerOperations`, `TroveManager`, MUSD itself, `SortedTroves`, `HintHelpers`, and the `PriceFeed` contract's own staleness logic, all real deployed bytecode. They cover the full lifecycle, that the real protocol rejects a sub-minimum holder, that trove debt includes the gas compensation, that `maxBorrowingCapacity` stays frozen until `refinance()`, that a stale oracle surfaces as a named error, and that exits stay open while entries are paused.

3.3

Contract sizes

```bash
forge build --sizes
```

All five deployed contracts are comfortably under the 24,576-byte limit.

CONTRACT	RUNTIME BYTES	MARGIN
`MarenVault`	**14,508**	10,068
`MarenEarn`	11,396	13,180
`MarenPay`	9,264	15,312
`MarenRelayHub`	7,934	16,642
`MarenRegistry`	4,742	19,834

`MarenForwarder` compiles to 3,999 bytes and is not deployed separately; deploying it alongside the hub would create two independent ways to forge a sender, only one of which would be allowlisted.

3.4

The headline number

The collateral needed to reach MUSD's 1,800 MUSD minimum net debt at a 250% collateral ratio is **0.079346 BTC, $5,004.50** at the oracle price of $63,071.83 recorded on 15 August.

That figure is **proven, not calculated in a document**. It is the output of `collateralRequiredFor()` in `packages/sdk/src/position.ts`, and a test asserts the result falls between 0.078 and 0.080 BTC. Anyone who clones the repository can run it.

Maren's floor is 0.001 BTC, and the real point is that it has **no minimum loan at all**: you can borrow 25 MUSD, which the protocol does not permit at any collateral level.

> WARNING
>
> Do not resurrect the "eighty times lower" or "79.3x" framing that earlier drafts used, and do not lead with the holder count. On-chain research showed that framing is a vanity metric: of MUSD's ~31M supply only ~4.6% was created by borrowing, ~46% farms the Stability Pool, and the top 20 holders (almost all contracts) hold ~99.5%. The credibility of this project rests on numbers that survive being checked, and a Mezo insider checks these in one query.

3.5

The chain facts the thesis rests on

MUSD is a DeFi-internal asset, not consumer money. ~27 troves, four whales holding ~80% of borrowed MUSD, nearly half the supply farming yield. That is not evidence of pent-up borrowing demand; it is evidence that the 2,000 MUSD minimum loan prices out small holders and that MUSD has no consumer surface. Maren is a bet that a consumer layer creates the demand, resting on Mezo's off-chain community rather than its trove count.

Re-run the counts on the day you send anything quoting them:

```bash
cast call 0x94AfB503dBca74aC3E4929BACEeDfCe19B93c193 \
  "getTroveOwnersCount()(uint256)" \
  --rpc-url https://jsonrpc-mezo.boar.network
```

3.6

Gas

Two different gas numbers are true at once, and an earlier draft of this document mislabelled one as the other, so both are stated here.

Mezo's **base fee** is 1,300,000 wei; the **gas price** a transaction actually pays is 1,462,500 wei. A 100,000-gas transaction therefore costs **1.4625×10^{-7} BTC** to send and **1.3×10^{-7} BTC** at the floor. Those parts are exact and do not move with the Bitcoin price.

At the $62,921.65 oracle price read on 16 August, that is **$0.0092** to send and $0.0082 at the base fee. A million sponsored transactions costs about **$9,200**. **$0.0092 is the gas-price figure**: a sponsor pays gas price, not base fee, so it is the correct number for what the relayer spends.

The earlier draft printed $0.0092 under the label “base fee” and explained the gap to $0.0082 as a Bitcoin-price difference. It is not: 1.3×10^{-7} BTC only reaches $0.0092 at a BTC price of $70,769, which nothing here uses. The gap was always base fee versus gas price. Quote the BTC figure, name which of the two it is, and say what price you converted at.

3.7

The applications

These figures are **recorded from the work that produced them** rather than re-executed today. The scripts that produce the first two are in the repository and can be re-run: `apps/web/scripts/verify-behaviour.mjs` and `apps/web/scripts/verify-contrast.mjs`.

Landing page. Next.js 15, **112 kB first load**, fully static, three shaders. **17 of 17 behavioural checks** pass, reduced-motion shader fallback, accordion ARIA, keyboard operation, focus rings, landmarks, and **8 of 8 contrast probes** pass with WebGL enabled, which is the case that matters, because a contrast measurement taken with the shader disabled measures a page nobody sees.

iOS app. Eight screens, **verified running in the iOS Simulator**: onboarding, Vault, and Pay were captured and inspected rather than assumed from a clean build.

3.8

The document set

```bash
CI=true pnpm docs:pdf
```

All eight documents build, 101 pages in total.

The same command also builds the demo-day deck, MRN-008. It is kept out of the table below because it is measured in slides rather than pages: 15 of them, one page each at 16:9. It takes a separate path through the build for the same reason, and the build refuses it if any slide overflows its frame.

DOC	DOCUMENT	PAGES
MRN-000	Grant Package Overview	4

DOC	DOCUMENT	PAGES
MRN-001	Investment & Grant Thesis	16
MRN-002	Technical Feasibility & Architecture	28
MRN-003	Product Specification	7
MRN-004	Security & Risk	9
MRN-005	Go-to-Market	9
MRN-006	Roadmap & Grant Milestones	9
MRN-007	Work Summary, this document	19
Total		**101**

The build is deterministic: dates are taken from each document's own frontmatter rather than from the clock, so rebuilding unchanged sources produces byte-identical files instead of a diff every time.

4

What is not verified, and what is not done

4.1

Matsnet is deployed; mainnet is not

Superseded on 15 August 2026, later the same day this document was written. Five contracts are deployed and source-verified on Mezo matsnet (31611), and `MAREN_ADDRESSES` in `packages/sdk/src/addresses.ts` carries their real addresses. `MarenRegistry.register`, `MarenVault.openPooledTrove`, `deposit` and `topUpCollateral` have all executed on-chain against real MUSD. Tx hashes are in `docs/DEPLOYMENTS.md`, along with the two findings that only appeared by running it, a pooled trove seeded at the protocol minimum can never let a user borrow, and `deleverage()` reported success while doing nothing.

Mainnet is still all zero addresses, and `isDeployed()` still exists so that every consumer confronts that rather than discovering it.

The qualification that matters is narrower than it was but has not gone away: everything in section 3 remains a property of code that has never held **a stranger's** money. The only funds it has touched are the author's own testnet BTC.

4.2

The relayer is not merged

`apps/relayer` has **214 tests passing** and is merged to `main`.

An adversarial review, including 71-mutation testing, found the **cryptographic core sound and the economic abuse layer broken**. The fixes are pushed and are **under re-verification**. Until that re-verification passes, the relayer is **not safe against a funded key** and must not be merged.

State it that way to anyone who asks. It is pending, not done. The temptation to describe a component as finished because the hard part is finished is exactly how a half-verified thing reaches production.

4.3

`MarenEarn`'s swap path has venues, but is not wired yet

The Stability Pool pays its gains in native BTC, and `MarenEarn`'s unit of account is MUSD. Converting between them needs a DEX. Mezo **does** have one: a BTC/MUSD volatile AMM and a MUSD/mUSDT stable AMM are live on mainnet (found in the MUSD holder set). The earlier claim that no Mezo DEX had been verified was wrong. What is true is that Maren has not yet integrated a router.

So `swapRouter` is `address(0)` at deployment and `harvest()` simply **holds** the BTC, counted in `totalAssets()` at the oracle price. Shareholders own it and it is not stranded, but the vault is then partly BTC-denominated and its share price moves with BTC. That is a real, deliberate exposure, and `ISwapRouter` is a seam, not an integration. Do not describe it as one.

4.4

The pooled vault carries real systemic risk

`MarenVault` holds one trove. A user's collateral is **segregated in accounting, not in custody**. If the MUSD protocol ever liquidated that trove, the loss would fall across all pooled users pro rata, including users whose own position was perfectly healthy.

The mitigations are distance from the cliff (no borrowing below 180%, internal liquidation at 140%, against the protocol's 110%), a per-user ledger, a deposit cap that widens on evidence rather than on schedule, an insurance buffer funded from the 5% liquidation penalty, and the rule that entries pause before exits do. They reduce the probability. They are not a guarantee, and the commitment in the roadmap, ship without the pooled vault if it cannot be made safe to our own satisfaction, is the honest backstop.

4.5

Everything is pre-audit

No third party has reviewed any of this. `MarenVault`, `MarenForwarder`, and `MarenRelayHub` are the audit scope: the first holds the funds, and the other two hold authority, the hub can make an arbitrary allowlisted call appear to come from any user who has signed anything, and it moves fees under user allowances.

4.6

Two environment limitations worth recording

Both cost real time to establish and both will be rediscovered by whoever works on this next, so they are written down rather than remembered.

Mezo's oracle cannot be forked. The BTC/USD feed is Skip Connect, a consensus-level `x/oracle` module in mezod rather than a deployed contract. `eth_getCode` at `0x7b7c000000000000000000000000000000000015` does return bytecode, but that bytecode is a stub whose only action is to call itself; on Mezo, mezod intercepts and the native handler answers. Inside Foundry's EVM there is no handler, so it recurses and reverts, taking `PriceFeed.fetchPrice()` and every borrow, adjust, and liquidate path with it. The fork suite therefore `vm.etch`es a stub at that address. **Everything else in the fork tests is real deployed bytecode**; only the consensus oracle underneath is substituted, because it cannot be replayed. The live precompile is verified out of band with `cast`.

foundry 1.4.3 cannot express Mezo in `[etherscan]`. It resolves that table by looking the chain id up in a built-in chain list, and Mezo is not in it, so *any* `[etherscan]` entry naming 31611 or 31612 makes `forge script` fail outright with `Error: Chain 31611 not supported` before it does anything else. Established by bisection. Verification therefore passes `--verifier blockscout --verifier-url ...` on the command line, which works today. Note the `api.` subdomain, `explorer.mezo.org/api` without it returns 404.

5

Two findings that did not survive verification

Two claims made during this work were **wrong and have been retracted**. Both were caught by re-verification before anything shipped, which is the point of recording them: the discipline is not decoration, it has already stopped two errors from reaching Mezo.

5.1

The dRPC claim

An early probe sweep showed `https://mezo.drpc.org` returning `-32601 "the method does not exist/is not available"` for `eth_call`, `eth_getLogs`, `eth_estimateGas`, and `eth_blockNumber`, and this was written down as a capability limit, that the endpoint serves `eth_chainId` and nothing else.

It was a transient outage. On re-test the same endpoint served the same methods correctly 10 out of 10 times about an hour later. A degraded backend behind a load balancer had reported “unsupported” rather than “unavailable”, which is indistinguishable from a genuine capability limit if you probe once.

Two consequences were kept, and they are worth more than the original finding was:

1. **Never certify or condemn an endpoint on a single probe.** Repeat it.
2. **Every client fails over across a list rather than pinning one URL.** A `-32601` on this chain is not proof of a missing feature.

No endpoint is denylisted anywhere in the codebase, and no upstream pull request makes any claim about dRPC. `packages/sdk/src/chains.ts` carries the corrected account in full. The last uncorrected copy was a comment in `packages/contracts/foundry.toml`, which is fixed in the same change that adds this document.

5.2

The mis-citation

The two MUSD documentation-versus-bytecode discrepancies, the borrower redemption-fee waiver that the deployed `TroveManager` does not implement, and the refinance fee that computes to roughly 0.02% rather than the documented 0.1%, were initially placed in the `mezo-org/musd` repository.

They are not there. `mezo-org/musd` does not state the borrower waiver at all; its closest line says the opposite. Both claims live in **`mezo-org/documentation`**, at `users/borrow/fees.md` and `users/borrow/index.md`. The underlying code observations are accurate, only the location was wrong, and the issue text was rewritten to quote the correct files.

A related correction went with it: `isAccountFeeExempt` is a **governance-managed allowlist** (`addFeeExemptAccount` is `onlyGovernance`), not a borrower mechanism. “Borrowers are exempt” and “fee-exempt accounts” are two different things.

> WARNING
>
> This is the one to not repeat in front of Mezo. Telling their team that a discrepancy is “in the musd repo” when it is in the documentation repo is a thirty-second correction that costs the credibility of the other nine things in the message, and the whole strategy of the outreach is that the findings are trustworthy. The prepared issue text already quotes the right files. Use it verbatim rather than paraphrasing from memory.

6

Upstream contributions, prepared and not submitted

Three pull requests and one issue are written, verified, and **pushed to Sele’s forks. Nothing has been opened against any upstream repository.** Each is one command to fire, and all four are in `docs/research/UPSTREAM-PRS.md` with the evidence.

1, a pull request against `wevm/viem`. Mezo mainnet’s default RPC host does not resolve, and the definition omits `multicall3` although Multicall3 is deployed at the canonical address, birth block 351760.

2, a pull request against `mezo-org/documentation`. Three RPC and explorer hosts that return NXDOMAIN, two of them inside copy-pasteable snippets that therefore fail at the first line.

3, a pull request against `mezo-org/documentation`. The Goldsky network slug is `mezo`, not the documented `mezo-mainnet`, so a manifest copied from the docs is rejected at deploy time.

4, an issue against `mezo-org/musd`. The two documentation-versus-bytecode discrepancies, framed as questions rather than as reports, because whether the docs or the contracts should change is the maintainers' call.

Read the retraction section of `UPSTREAM-PRS.md` before firing number 4. That is where the mis-citation was caught, and the framing changed as a result.

The viem fix is the highest-value one to Mezo, because a missing `multicall3` silently degrades every batched read for every wagmi user on the chain rather than failing loudly.

7

What to do next

In priority order.

1. Deploy to matsnet. Nothing else in this document is worth much until an address exists.

```bash
forge script script/Deploy.s.sol \
  --rpc-url mezo_testnet --broadcast --account maren-deployer \
  --verify --verifier blockscout \
  --verifier-url https://api.explorer.test.mezo.org/api
```

The verifier goes on the command line, not in `foundry.toml`, for the reason in section 4.6. Then open the pooled trove as `KEEPER_ROLE` with hints computed off-chain, and update `packages/sdk/src/addresses.ts` from `deployments/<chainid>.json`. The post-deploy checklist in `packages/contracts/README.md` is the one to work through, not this paragraph.

2. Merge the relayer once re-verification passes, and not before. It holds a funded hot key. The economic abuse layer is the part that was broken, the fixes are pushed, and the re-verification is the gate. A merge before that gate is the single most expensive thing that could be done this week.

3. Ask for the audit introduction. An introduction to whoever audited MUSD is worth more than the grant money, because an auditor already holding the protocol context can review a contract sitting on top of it far more cheaply than one starting cold. It is also cheap for Mezo to give, which is why it should be asked for early and directly rather than saved.

4. Open the four upstream contributions. They are one command each. Opening them before the reply goes out turns “I’ve drafted PRs and can send them whenever” into “they’re open”, which is a materially stronger sentence, and it costs nothing.

5. Reply to Judy. The draft is `docs/research/OUTREACH-REPLY.md`. Two things to settle first: re-confirm the trove count on the day it is sent, because it moves, and decide whether the repository is public before offering a link. Attach the thesis and the feasibility papers.

The reply is last in this list only because its content improves as the items above land. It is not blocked on any of them, and the correspondence should not be allowed to go cold while a deployment is being polished.

8

The shape of it

What exists is a complete, tested system that is now live on matsnet: five contracts with 236 local tests and 8 fork tests against the real protocol, deployed and source-verified, two applications, an SDK, a relayer one verification away from merging, and seven documents.

What does not exist is a mainnet address, an audit, or a single satoshi of anyone else’s money. The pooled trove on matsnet holds 0.049 BTC of the author’s own testnet collateral, and the first thing it did was refuse a borrow, correctly.

The distance between those two paragraphs is an audit, and everything in section 4 should be said out loud before anyone else has to find it.

An account of the repository at github.com/SeventhOdyssey71/marenfinancè as of 15 August 2026, written for the person who will be asked to defend it.

Every figure in section 3 was re-executed on the day of writing rather than copied from an earlier document, and the command that produces it is printed alongside. Figures that could not be re-executed are marked as recorded rather than as verified.

Sections 4 and 5 exist because a summary that only lists successes is worth nothing to the person holding it in a meeting. Anything not stated here as verified should be assumed not to exist.